

comemo

原理深度介绍与源码阅读指南

Incremental Computation through Constrained Memoization

受约束记忆化 · 访问跟踪 · 调用树 · 过程宏 · 增量计算

技术栈： Rust 2024 · proc-macro (syn/quote) · SipHash-1-3 · Slab

源码版本： comemo 0.5.0 (github.com/typst/comemo)

整理： NovaForge

最后修订： 2026 年 8 月 1 日

项目概览

一句话定位

comemo 是 Typst 排版引擎的增量计算核心库：通过**受约束记忆化（constrained memoization）**，让任意纯函数的缓存失效粒度从“整个输入”细化到“实际读到的数据”，从而支撑 Typst 的毫秒级增量重排版。

仓库布局

路径	行数	职责
src/lib.rs	118	crate 文档与模块导览；公开 API re-export；internal 实现细节模块
src/track.rs	338	Tracked / TrackedMut 包装与 Track / Call / Sink / Surfaces trait
src/input.rs	270	Input 输入抽象、MergedSink、multi! 元组输入
src/constraint.rs	166	Constraint 约束记录与 CallSequence 去重序列
src/memoize.rs	192	memoize() 主调度、Cache、全局逐出
src/tree.rs	418	CallTree 前缀树：查找/插入/剪枝 + 单元测试
src/accelerate.rs	63	约束验证加速器（epoch 式清理）
src/hash.rs	11	128 位 SipHash-1-3 预哈希
src/testing.rs	21	测试用 hit/miss 标志（testing feature）
macros/src/lib.rs	216	两个属性宏的入口与完整契约文档
macros/src/memoize.rs	174	#[memoize] 展开：校验 + 代码生成
macros/src/track.rs	410	#[track] 展开：Call 枚举/surface/wrapper 生成
examples/	104	basic.rs 最小示例、calc.rs 增量解释器示例
tests/tests.rs	733	13 个集成测试 + 1 个 quickcheck 性质测试

内容导航

章节	内容
一、总体原理与概念模型	基础记忆化的局限；三大机制；reorderably deterministic 等正确性契约；三类参数
二、跟踪层与输入抽象	Tracked / TrackedMut 结构；四个 trait 契约；Input 五件套；MergedSink；加速器；预哈希
三、约束记录与缓存内核	Constraint 双通道；memoize() 主流程；CallTree 四表结构；查找/插入/剪枝；age 逐出
四、过程宏层	#[memoize] 展开流程与生成骨架；#[track] 四个生成物；校验规则全表
五、示例、测试与阅读指南	两个示例逐行分析；测试版图；三遍阅读法
附录	关键类型速查表；术语表

目录

项目概览	2
一、总体原理与概念模型	4
基础记忆化 vs 受约束记忆化	4
三大机制的交互（概念层）	4
正确性前提（用户契约）	5
三类参数（Kinds of arguments）	5
本章源码精读路线	5
易错点与设计权衡	6
二、跟踪层与输入抽象	7
Tracked<'a, T, C> 的结构与 Deref 机制	7

TrackedMut 与 Tracked 的差异	7
四个 trait 的方法与契约	7
Tracked 的 variance 问题与 Chain 解法	7
Input trait 五件套与三个实现	8
嵌套 memoize 的正确性枢纽：命中也必须转发	8
MergedSink：嵌套调用的 sink 串联	8
multi! 宏：元组输入（至多 12 元）	8
accelerate.rs：约束验证加速器	9
hash.rs：128 位预哈希	9
本章源码精读路线	9
易错点与设计权衡	9
三、约束记录与缓存内核	11
Constraint<C> 的双通道结构	11
CallSequence<C>：为插树优化的去重序列	11
memoize() 主流程逐段剖析	11
CallTree：四张表 + NodeId 编码	12
get：用 oracle 重放校验	12
insert：前缀对齐 + 开新路径	12
retain：自叶向根剪枝	13
Cache / CacheEntry 与 age 逐出	13
calc 例子完整走一遍	13
本章源码精读路线	13
易错点与设计权衡	13
四、过程宏层（comemo-macros）	15
#[memoize] 的展开流程	15
#[track] 的展开流程	15
utils.rs 的辅助	16
手写等价展开示例（示意）	17
本章源码精读路线	17
易错点与设计权衡	17
五、示例、测试体系与源码阅读指南	18
examples/basic.rs 逐行讲解（最小上手例子）	18
examples/calc.rs 的 Hit/Miss 逐条分析	18
src/testing.rs 与 testing feature 的联动	18
tests/tests.rs 测试版图	18
源码阅读指南：三遍阅读法（本指南的压轴）	19
易错点	20
附录 A：关键类型与函数速查表	21
A.1 公开 API（用户侧）	21
A.2 运行时核心（internal）	21
A.3 宏展开关键点	22
附录 B：术语表	23

一、总体原理与概念模型

定位：comemo 是什么、解决什么问题

- comemo 是 Typst 排版引擎的增量计算（incremental computation）核心库，一句话定位：“Incremental computation through constrained memoization”（src/lib.rs:2、README.md:5）。
- 它要解决的问题：基础记忆化（memoization）把“整组参数”当作缓存键，粒度过粗——“lacks the necessary granularity”（README.md:14-15）。以 .calc 语言解释器为例，evaluate 需要整个文件集合 &Files 作为输入，于是“a change to any file invalidates all memoized results”（README.md:49-51），任何一个无关文件改动都会让全部缓存失效。
- 受约束记忆化（constrained memoization）的核心思想：缓存条目不只绑定参数哈希，还绑定调用期间实际观察到的访问记录（约束）；只要这些被观察到的部分没变，即使输入整体变了也能命中（README.md:53-63、src/lib.rs:48-62）。
- crate 分两层：comemo 本体提供运行时（src/lib.rs:91-97 的七个模块），comemo-macros 提供 #[memoize] 与 #[track] 两个属性宏（macros/src/lib.rs:133-139、macros/src/lib.rs:211-216）。用户接入只需三步：给函数加 #[memoize]、给 impl 块加 #[track]、把参数包进 Tracked（README.md:56-58）。

基础记忆化 vs 受约束记忆化

- 基础记忆化：函数“caches its return values so that it only needs to be executed once per set of unique arguments”（README.md:12-13）。缓存键 = 全部参数的哈希。
- .calc 反例：脚本由数字加法与 eval <path> 语句构成（README.md:17-22；alpha.calc = "2 + eval beta.calc" 等），解释器递归读取依赖文件（examples/calc.rs:40-49）。若把整个 Files 哈希为键，改 gamma.calc 也会使只依赖 beta.calc 的 alpha.calc 结果失效。
- 受约束记忆化：把“输入值”换成“输入值 + 本次执行实际读到了什么”。examples/calc.rs:28-29 演示：修改 gamma.calc 后 evaluate("eval alpha.calc", ...) 仍 [Hit]，因为 gamma.calc 未被 alpha.calc 引用；而 examples/calc.rs:34-35 修改 beta.calc 后 [Miss]。
- 另一个粒度收益：嵌套命中。examples/calc.rs:21-23——顶层字符串没见过（顶层 miss），但内部 "2 + 3" 的求值不会重算（内层 hit）。

基础记忆化：缓存键 = 全部参数哈希

任一输入字节变化 → 全部失效

⇓ constrained memoization：键 + 约束

参数哈希 key

+

(tracked 调用, 返回哈希) 约束序列

→

重放验证仍成立 → 命中

图 1 基础记忆化与受约束记忆化的失效粒度对比

三大机制的交互（概念层）

1. 访问跟踪（access tracking）：#[track] 让类型的方法可被观测；Tracked<T> 容器在方法调用时把“调用了哪个方法、传了什么参数、返回值的哈希”记录到一个 sink（src/track.rs:12 的 Track trait、src/track.rs:59 的 Sink、src/track.rs:162 的 Tracked）。
2. 约束记录与验证（constraint recording/validation）：一次 memoized 调用产生的调用序列被记录为 Constraint（src/constraint.rs:17；验证入口 Constraint::validate，src/constraint.rs:35）。下次调用时用当前输入重放验证：约束仍成立 → 命中；不成立 → 重算。底层由调用树（call tree）高效存储多组调用序列（src/tree.rs:15-26）。
3. 变更重放（mutation replay）：对 TrackedMut<T> 参数的可变调用也被记录；缓存命中时函数体不执行，comemo 把记录的变更重放到参数上，保证可观察副作用一致（macros/src/lib.rs:39-42；缓存条目中有“Mutable tracked calls that must be replayed”，src/memoize.rs:137-144）。

交互时序（伪代码）：

```
memoized_call(args):
    key = hash(hashed_args)           // 128 位哈希, src/hash.rs:7
    if 缓存中存在 key 且约束重放验证通过:
        replay 记录的 mutable 调用    // src/memoize.rs:56-59
        return 缓存结果                // hit
```

```

else:
    attach 约束 sink → 执行函数体 // 期间 tracked 调用被记录
    存入缓存                               // miss

```

正确性前提（用户契约）

以下契约全部出自宏文档（`macros/src/lib.rs:44-90`），是 comemo 正确工作的前提：

- **Hash 必须喂全信息**：参数的 Hash 实现 “must feed all the information they expose to the hasher”，否则缓存结果可能被错误复用（`macros/src/lib.rs:47-50`；tracked 方法的返回值同样要求，`macros/src/lib.rs:170-173`）。
- **唯一可观察不纯性是 TrackedMut 变更**：“The only observable impurity memoized functions may exhibit are mutations through `TrackedMut<T>` arguments”（`macros/src/lib.rs:52-54`）。comemo 能阻止普通可变参数，但无法检测所有不纯来源（全局状态、IO、随机数等），这是用户的责任。tracked 方法同理：不可观察的内部可变性（idempotent 的 interior mutability）被允许（`macros/src/lib.rs:157-168`）。
- **reorderably deterministic（可重序确定）**的精确定义（`macros/src/lib.rs:57-84`）。设同一 memoized 函数的两次执行 A 与 B：
 - In-order deterministic（按序确定）（`macros/src/lib.rs:61-69`）：若 A 与 B 的前 N 次 tracked 调用及其结果相同，则第 N+1 次调用也必须相同。对确定性函数这很自然，但实践中过严——例如内部使用多线程的函数可能乱序发起 tracked 调用，整体结果仍是确定的。
 - Reorderably deterministic（可重序确定）（`macros/src/lib.rs:71-78`）：若对于 A 的前 N 次调用，B 的调用序列中某处存在匹配调用（参数相同、返回值相同），则 A 发起的第 N+1 次调用也必须出现在 B 的调用序列的某处。
 - 放宽理由：这是 in-order determinism 的松弛版，允许缓存“内部多线程但对外确定”的函数（`macros/src/lib.rs:74-78`）。
 - 违反后果：debug 模式可能 panic 提醒；release 模式结果仍正确，但缓存可能失效（`macros/src/lib.rs:80-84`）。
- 输出须为 Send + Sync（存入全局缓存，`macros/src/lib.rs:86-87`）；参数不能用解构模式（`macros/src/lib.rs:89-90`）。

易错点：不能用“结果对不对”判断程序是否满足契约——release 下契约违反永远表现为性能问题（缓存静默失效）而非正确性问题。调试时应开 debug 断言跑一遍。

三类参数（Kinds of arguments）

出自 `macros/src/lib.rs:28-42`：

类别	形式	语义
Hashed（默认）	普通参数	哈希为高质量 128 位哈希作缓存键（ <code>macros/src/lib.rs:31-32</code> ）
Immutably tracked	<code>Tracked<T></code>	细粒度访问跟踪；只要差异未被观察到，T 值不同也可命中（ <code>macros/src/lib.rs:34-37</code> ）
Mutably tracked	<code>TrackedMut<T></code>	可在 memoized 函数内安全变更参数；命中时重放全部变更；可变 tracked 方法不能有返回值（ <code>macros/src/lib.rs:39-42</code> ）

- 可变参数只能走 tracking（不能哈希），以便命中时重放副作用（`macros/src/lib.rs:148-151`）。
- `Tracked` / `TrackedMut` 由 `track()` / `track_mut()` 产生（`src/track.rs:25`、`src/track.rs:31`）。

本章源码精读路线

自顶向下、先契约后实现：

1. `README.md`（全文）：建立问题直觉。重点：12-15（基础记忆化定义与粒度缺陷）、17-51（`.calc` 动机例子）、53-63（三步接入与“依赖不变即命中”的承诺）。
2. `examples/calc.rs`（全文，约 65 行）：把概念对应到代码。main 中四处 `[Miss]` / `[Hit]` 注释就是约束验证的行为规格说明。
3. `src/lib.rs:1-88`（模块级文档）：与 README 几乎同文；注意：91-97 的模块划分与 111-117 的 `internal` 模块（“Do not rely on them”）。
4. `macros/src/lib.rs:15-132`（`#[memoize]` 文档）：契约核心。reorderably deterministic 的两条性质在 61-69 与 71-78，务必对照原文逐句读。
5. `macros/src/lib.rs:135-189`（`#[track]` 文档）：tracked 方法的限制清单。

易错点与设计权衡

- **纯度责任在用户**：comemo 只能用类型系统挡住显式可变参数，挡不住全局状态、IO、随机数、可观察的内部可变性。这类 bug 表现为缓存错误命中，极难排查。
- **Tracked 不是值比较**：命中条件是“差异未被观察到”，而非“值相等”。`T` 整体从不哈希，只有被调用方法的参数与返回值进入约束。
- **设计权衡：调用序列作为正确性凭证**。comemo 选择“记录(调用, 返回哈希)序列并重放验证”而非维护显式依赖图。好处：用户零注解成本、嵌套调用天然分层；代价：要求 reorderably deterministic，且验证成本与调用序列长度相关（为此引入加速器与调用树做内部优化）。
- **与 salsa 等按需增量框架的定位对比**：comemo 是 memoization 的增强——以函数为缓存单位、以访问跟踪缩小失效范围，API 只有两个属性宏，无显式 query 数据库概念。**[推断]** 相比 salsa 的 query-group + 依赖图 + revision 计数模型，comemo 更接近“自动依赖发现 + 按需重放验证”；salsa 的具体机制请以 salsa 自身文档为准。
- **缓存是全局且需手动驱逐**：条目存入全局缓存，靠 `evict(max_age)` ([src/memoize.rs:101-106](#)) 按 age 清理。长时间运行的程序若从不调用 `evict`，内存只增不减。

二、跟踪层与输入抽象

定位：把“对值的普通方法调用”转化为“可记录、可重放的调用事件”

跟踪层 (tracking layer) 涉及四个文件：`src/track.rs` 定义 `Tracked / TrackedMut` 包装类型与 `Track / Call / Sink / Surfaces` 四个 trait；`src/input.rs` 定义 `Input` trait 统一输入抽象，并提供 `MergedSink` 串联嵌套调用的 sink 链、`multi!` 宏支持元组输入；`src/accelerate.rs` 提供约束验证加速器；`src/hash.rs` 提供全库统一的 128 位预哈希。

`Tracked<'a, T, C>` 的结构与 `Deref` 机制

定义于 `src/track.rs:162-176`，三个字段（均 `pub(crate)`）：

- `value: &'a T` (`src/track.rs:167`) ——被跟踪值的不可变引用；
- `sink: Option<&'a dyn Sink<Call = C>>` (`src/track.rs:173`) ——调用事件的接收方，初始为 `None`，在 `memoized` 函数序言中被设为栈上存储的约束 sink（字段文档 `src/track.rs:168-172`）；
- `id: usize` (`src/track.rs:175`) ——验证加速器的唯一 ID，由 `accelerate::id()` 分配。

`Deref` 到 `surface` 类型：`impl Deref for Tracked` (`src/track.rs:181-191`)，`Target = T::Surface<'a>`。注释 (`src/track.rs:178-180`) 点明设计意图：自动 `deref` 使 `#[track]` 标注的方法可用，而 `T` 的其他方法不可达——`surface` 类型是访问的唯一入口，因此所有访问都能被拦截记录。

Copy/Clone 语义：`Tracked` 是 `Copy` (`src/track.rs:203`)。它只含共享引用、`Option<&dyn Sink>` 和 `usize`，可以像普通引用一样自由传递、多处共用同一跟踪上下文。

`TrackedMut` 与 `Tracked` 的差异

- 定义于 `src/track.rs:223-235`：`value: &'a mut T + sink`，没有 `id` 字段——可变调用有副作用，不能用加速器缓存返回哈希。
- 实现 `Deref` 与 `DerefMut` (`src/track.rs:277-297`)，后者经 `surface_mut_mut` 暴露可变 `surface`，使可变方法也能被拦截。
- 三个关联函数刻意做成关联函数而非方法，避免与 `T` 上同名方法冲突（注释 `src/track.rs:241-243`）：`downgrade` (`src/track.rs:246`，可变降级为不可变，此时才分配 `accelerator id`)、`reborrow` / `reborrow_mut` (`src/track.rs:259`、`src/track.rs:272`)。
- `TrackedMut` 不是 `Copy`（含 `&mut`），必须线性传递以保证 `call_mut` 副作用只发生一次。

四个 trait 的方法与契约

- `Track` (`src/track.rs:12-56`)：可跟踪类型。关联类型 `type Call: Call` 是该类型所有可能被跟踪调用的枚举；`call(&self, &Call) -> u128` 执行调用并返回结果哈希；`call_mut(&mut self, &Call)` 执行可变调用。四个便捷构造器：`track()` (:25)、`track_mut()` (:31)、`track_with(sink)` (:37)、`track_mut_with(sink)` (:50)。
- `Sink` (`src/track.rs:59-68`)：调用事件的目的地址，要求 `Send + Sync`。`emit(call, ret: u128) -> bool` 的返回值含义（文档 `src/track.rs:63-66`）：返回 `false` 表示该调用已被去重，调用方应避免再向层级更高的其他 sink 发送。`&S` 的 `blanket impl` (:70-76) 让 sink 引用本身也是 sink。
- `Call` (`src/track.rs:79-82`)：一次被跟踪调用的抽象，要求 `Clone + PartialEq + Hash + Send + Sync`；唯一方法 `is_mutable()`。() 的 `impl` (:85-89，恒为 `false`) 供纯哈希输入占位。
- `Surfaces` (`src/track.rs:92-121`)：`surface` 类型族。两个 GAT：`Surface<'a>` 与 `SurfaceMut<'a>`；三个访问器 `surface_ref` / `surface_mut_ref` / `surface_mut_mut`。`surface` 类型由 `#[track]` 宏生成（见第四章）。

`Tracked` 的 variance 问题与 Chain 解法

文档注释 `src/track.rs:125-161`（「## Variance」节）：默认类型参数 `C = <T as Track>::Call` 中 `T` 出现在关联类型投影里，由于编译器限制，此时 `Tracked<'a, T>` 对 `T` 是 **invariant**（不变）的。

- 后果：无法构造可用的 `tracked` 类型“链”，如 `struct Chain<'a> { outer: Tracked<'a, Self>, data: u32 }` (`src/track.rs:131-136`) ——invariance 阻止生命周期缩短。而该模式有用，例如检测 `memoized` 递归算法中的环。
- 解法 (`src/track.rs:144-152`)：手动指定调用类型 `Tracked<'a, Self, <Chain<'static> as Track>::Call>`，切断 `C` 对 `'a` 的依赖使 `T` 恢复协变。`'static` 的用意：让编译器确信关联约束类型不依赖 `'a`——事实上所有约束都是 `'static` 的。

Input trait 五件套与三个实现

定义于 `src/input.rs:13-41` :

- `type Call: Call (:16)` —— **该输入所有 tracked 部分可能产生的调用枚举**;
- `type Storage<S: Sink<Call = Self::Call> + 'a>: Default (:20)` —— **存放“合并后 sink”的外部存储类型**;
- `fn key<H: Hasher>(&self, &mut H) (:23)` —— **只对非 tracked 部分哈希, 作为缓存键**;
- `fn call(&self, &Call) -> u128 (:29)` —— **在 tracked 部分上重放调用并返回结果哈希**;
- `fn call_mut(&mut self, &Call) (:34)` —— **重放可变调用, 无返回哈希**;
- `fn attach<S>(&mut self, storage: &'a mut Storage<S>, sink: S) (:38-40)` —— **把给定 sink 整合进输入的 tracked 部分**。

实现	Call	Storage	key	call / call_mut / attach
<code>T: Hash (:43-68)</code>	<code>()</code>	<code>()</code>	真正哈希自身	<code>call</code> 恒返回 0 ; <code>call_mut</code> 空操作; <code>attach</code> 空操作
<code>Tracked<'a, T> (:70-116)</code>	<code>T::Call</code>	<code>Option<MergedSink></code>	空操作	<code>call</code> 走加速器 + sink 转发 (:81-102) ; <code>call_mut</code> 空操作; <code>attach</code> 包进 <code>MergedSink</code> (:110-115)
<code>TrackedMut<'a, T> (:118-153)</code>	<code>T::Call</code>	同上	空操作	<code>call</code> 直接执行 + 转发; <code>call_mut</code> 真执行副作用并 emit (:138-143)

要点: tracked 输入的 key 为空——它们不参与缓存键, 而是通过约束验证缓存有效性; 纯哈希输入则相反, 只建键、无约束。两者职责正交互补。

易错点: Storage 关联类型为什么需要外部传入的 &mut Default 存储 (`src/input.rs:20`、`:38-40`) : attach 要把生命周期为 'a 的合并 sink 存进 `sink: Option<&'a dyn Sink>` 字段, 但 `MergedSink` 是函数局部值, 不能引用局部变量。解法是让调用方 (宏生成的序言) 在栈上持有 `Storage<S>`, attach 用 `storage.insert(...)` 把值放进外部存储再取引用——存储所有权在调用方帧上, 引用才能活到 'a。

嵌套 memoize 的正确性枢纽: 命中也必须转发

`src/input.rs:93-99` 的原注释 (Tracked 的 call 内):

The call method is used during the constraint validation tree traversal. It's crucial that we also send calls to the outer sink here so that the outer sink observes the calls when we have a cache hit. We do not replay the constraints in another way.

含义: call 在约束验证树遍历时被用来重放调用; 即使加速器和 memoize 缓存命中 (没有真正执行函数体), 也必须把 (调用, 返回哈希) emit 到外层 sink。这是**嵌套 memoize 正确性的关键**: 外层函数的约束必须记录“我这次执行间接依赖了内层 tracked 值的哪些调用”, 否则外层缓存命中时这些依赖丢失, 下次验证就无法发现内层值已变。comemo 没有第二条约束重放路径——验证路径与记录路径是同一条 emit 链路。

MergedSink : 嵌套调用的 sink 串联

- **定义** `src/input.rs:155-159` : `prev: Option<&'a dyn Sink> + sink: S`, Copy + Clone。
- **emit 实现** (`src/input.rs:168-176`) : `self.sink.emit(call.clone(), ret) && prev.emit(call, ret)` —— **短路逻辑**: 当前 sink 若已去重 (返回 false), 就不再向 prev 传播 (注释 :170-171)。
- **串联场景**: 外层 memoized 函数 f 把 constraint sink attach 到 tracked 输入; f 内部调用内层 memoized 函数 g 并传入该输入, g 的序言再次 attach, 形成 `MergedSink { prev: 外层 sink, sink: g 的 sink }`。于是内层的一次 tracked 调用同时进入两级约束, 两级缓存各自都能验证。

multi! 宏: 元组输入 (至多 12 元)

- **入口** `src/input.rs:182-251`, 实例化支持 0 至 12 元组 (`src/input.rs:255-270`)。 `Multi<T>(pub T) (:180)` 是元组输入的包装。
- **每个实例生成三样东西**:
 1. `impl Input for Multi<(A, B, ...)> (:184-213)` : `Call = MultiCall<A::Call, B::Call, ...>` ; key 逐元素哈希; `call / call_mut` 按 `MultiCall` 变体分发; `attach` 把 sink 包上 `MappedSink::<idx, _>` 后逐元素下传。
 2. `pub enum MultiCall<A, B, ...> (:217-219)` : 变体以类型参数命名, `is_mutable` 委派给内部调用。

3. `pub struct MappedSink<const I: usize, S>(S) (:232)`：把元素级 `emit` 映射为整体级 `MultiCall` 变体再发给外层 `sink`。const 泛型 `I` 仅为区分同一元组中同类型元素的不同 `impl`。
- 效果：混合输入（有的参数可哈希、有的 `tracked`）被拆解——哈希部分进缓存键，`tracked` 部分各自 `attach` 合并 `sink`。

accelerate.rs：约束验证加速器

动机：约束验证时需要 `tracked` 值重复重放所有记录的调用以获得返回哈希；这些 `tracked` 方法可能昂贵（如文件读取）。加速器把“调用哈希 → 返回哈希”缓存起来，同一 `tracked` 实例在一次验证周期内同一调用只执行一次（使用点 `src/input.rs:82-91`）。

数据结构：

- `static ACCELERATORS: RwLock<(usize, Vec<Accelerator>)>` (`src/accelerate.rs:7`) ——全局表，元组首元素是 `offset`；
- `static ID: AtomicUsize (:10)` ——单调递增的 ID 源；
- `type Accelerator = Mutex<FxHashMap<u128, u128>>` (`:15`)。

ID / offset / epoch 式清理（三个函数配合）：

1. `id() (:18-21)`：`ID.fetch_add(1)` 发新 ID。每个 `Tracked` 构造时拿一个（`src/track.rs:26`）。
2. `evict() (:24-33)`：每次 `memoize` 缓存逐出时调用。把 `offset` 更新为当前 ID 值（`:29`），并清空所有现存加速器但保留已分配内存（`:32`）。由于 ID 单调增，旧 ID `id < offset` 从此全部失效——不必逐个删除，一次 `offset` 推进即完成“epoch 切换”。
3. `get(id) (:36-54)`：`id.checked_sub(offset)` 把全局 ID 换算为 `vec` 下标；旧 epoch 的 `id` 直接返回 `None`，退化为真实调用。下标越界则 `drop` 读锁 → `resize` → 重拿读锁并再次 `checked_sub` 校验（`:43-50`）——因为放锁期间另一线程可能 `evict` 推进了 `offset`（注释 `:47-49`）。最后经 `RwLockReadGuard::map` 返回映射到单个 `Accelerator` 的读守卫。
4. `resize (:57-62)`：`#[cold]`，写锁下惰性补槽位。

hash.rs：128 位预哈希

`pub fn hash<T: Hash>(value: &T) -> u128` (`src/hash.rs:7-11`)：用 `SipHasher13` 的 128 位变体对任意可哈希值做预哈希。全库所有“值 → u128”的哈希都走它：加速器键/值、约束中的返回哈希、缓存键。128 位宽度让冲突概率可忽略，使 `comemo` 可以把“哈希相等”当作“值相等”来构建整条增量验证链（以哈希代值，避免存储/比较任意大对象）。

本章源码精读路线

建议顺序：`track.rs` → `input.rs` → `accelerate.rs` → `hash.rs`。

1. `src/track.rs:12-56` (`Track trait`)：`call` 返回的是“结果的哈希”而非结果本身——全库“以哈希代值”的第一次出现。注意 `track()` 里 `id: accelerate::id()` 而 `track_mut()` 没有 `id` 的不对称。
2. `src/track.rs:59-89` (`Sink 与 Call`)：`emit` 返回值的重短路语义是 `MergedSink` 正确性的前提。
3. `src/track.rs:92-121` (`Surfaces`)：GAT 声明方式；访问器以 `Tracked` 整体为参数（宏生成的 `surface` 方法需要 `sink` 和 `id`）。
4. `src/track.rs:123-211` (`Tracked + Variance 文档`)：`Variance` 文档逐行读 `Chain` 两个例子。
5. `src/track.rs:213-297` (`TrackedMut`)：三个关联函数为什么不做成方法。
6. `src/track.rs:309-339` (`to_parts_*`)：`memoize` 主流程用它们拆出 `value` 与 `sink`。
7. `src/input.rs:13-41` (`Input trait`)：精读每个成员的文档注释，`Storage` 与 `attach` 是最难懂的设计点。
8. `src/input.rs:70-116` (`Tracked impl`)：本章核心。`call` 分两层读：加速器查表（`:82-91`）→ `sink` 转发（`:93-99` 的注释必须逐字读）。
9. `src/input.rs:118-176` (`TrackedMut + MergedSink`)：对比两个 `call_mut`；`MergedSink::emit` 的 `&&` 短路。
10. `src/input.rs:178-270` (`multi! 宏`)：只读一个实例理解变体分发即可。
11. `src/accelerate.rs` (全文 63 行)：按 `id()` → `get()` → `evict()` → `resize()` 顺序读；`get` 的放锁-扩容-重取-再校验是唯一的并发精巧处。
12. `src/hash.rs` (全文 11 行)：确认 `SipHasher13` 与 128 位输出。

易错点与设计权衡

- `cache hit` 也必须向 `sink` 转发调用（`src/input.rs:93-99`）：最易误解处。以为“加速器命中 = 不用 `emit`”是错的——外层约束依赖这些事件做验证。
- 加速器的并发设计：先读锁、放锁、`resize`、再读锁、再校验（`src/accelerate.rs:39-51`）。`offset` 设计用“单调 ID + epoch offset”把逐出从 $O(n)$ 删除变为 $O(1)$ 比较，`evict` 清表时保留容量避免反复分配。
- `Sink::emit` 返回值的短路语义是性能优化而非可选项（`src/track.rs:63-67`）：忽略返回值不会错但会重复记录。

- **Tracked 是 Copy 而 TrackedMut 不是**：不可变跟踪上下文可多处共享；可变路径必须线性传递。
- **invariance 陷阱**：写 tracked 链会莫名报错，解法是显式给 `<Chain<'static> as Track>::Call`；初学者容易以为是生命周期标注错误。
- **key 与约束的分工**：纯哈希输入只进缓存键、tracked 输入只进约束。误以为 tracked 值也进键会导致把不可哈希的大对象塞进键。

三、约束记录与缓存内核

定位：comemo 的心脏——约束如何被记录、缓存命中如何被验证

宏生成的代码只是脚手架，真正把 constrained memoization 落到运行时的就是这三个文件：`src/constraint.rs` 在一次函数执行期间记录所有 tracked 调用及其返回值哈希，形成约束 (Constraint)；`src/memoize.rs` 是主调度器——查缓存、挂约束、执行、写回，并管理每个被记忆函数专属的 Cache 与全局逐出注册表；`src/tree.rs` 提供核心数据结构 CallTree，把“同一输入哈希 + 一串 (调用, 返回哈希) 前缀链”组织成一棵前缀树 (trie)，使命中与否取决于 tracked 依赖的返回值是否不变。

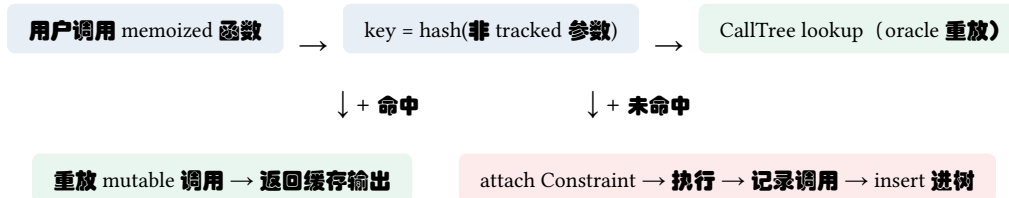


图 2 memoize() 主流程：命中与未命中两条路径

Constraint<C> 的双通道结构

`src/constraint.rs:21-27`：

```

ConstraintRepr<C> {
    immutable: CallSequence<C>, // 只读调用：要去重、要插树
    mutable: Vec<C>,           // 可变调用：只按序收集，不进树
}
  
```

- 分离点在 `Sink::emit (src/constraint.rs:69-76)`：`call.is_mutable()` 为真 → 压入 mutable 并返回 true；否则交给 `immutable.insert(call, ret)`。
- 为什么可变调用单独收集：可变调用没有返回值可哈希，无法参与“返回哈希是否一致”的校验，但它们有副作用。命中时函数体根本不执行，副作用必须重放——所以 mutable 随 CacheEntry 一起存进树 (`src/memoize.rs:137-144`)，命中时逐条 `input.call_mut(call)` 重放 (`src/memoize.rs:56-59`)。
- `Constraint::take()` (`src/constraint.rs:48-51`) 在插入缓存前用 `mem::take` 整体取出两个通道，保证 Constraint 是“一次性”的。
- 对外还暴露 `validate()` (`src/constraint.rs:34-46`)：对任意 Track 值重放 immutable 调用、逐个比对返回哈希。

CallSequence<C>：为插树优化的去重序列

字段 (`src/constraint.rs:82-89`)：`vec: Vec<Option<(C, u128)>>` (按首次出现顺序，Option 支持就地取走)、`map: FxHashMap<u128, usize>` (调用哈希 → 下标)、`cursor: usize (next 游标)`。

- 去重插入 (`src/constraint.rs:96-119`)：以 `hash::hash(&call)` 为键查 `map`：`Vacant` → 追加；`Occupied` → 去重，且 debug 构建下额外检查同一下标处 `ret != ret2` → `panic!("comemo: found differing return values. is there an impure tracked function?") (:111-117)`。这是第一道纯性防线：同一调用参数在同一轮执行里给出不同返回值。
- `next` 游标语义 (:122-130)：顺序消费，插入新树节点时使用。
- `extract` 抢读语义 (:132-139)：按调用哈希取走返回哈希 (该槽位之后 `next` 不再产出)。沿已有树路径走时使用。
- 于是 CallTree 的 insert 可以交错调用 `extract` (对齐已有路径) 与 `next` (消费剩余、开新路径)——这正是前缀树插入算法对序列容器的要求。

memoize() 主流程逐段剖析

`src/memoize.rs:17-93`。签名要点：`cache` 是宏为每个函数生成的 static；(storage, constraint) 必须从调用点外部传入 (注释 :21-24：只有这样才有足够长的生命周期把约束 attach 进 input)。

1. `enabled` 旁路 (:37-44)：`!enabled` 时直接执行返回，不查不写缓存。
2. 128 位 key 哈希 (:47-51)：`SipHasher13` 对 `input.key()` 做 `finish128()`。tracked 部分不进 key——它由树里的约束链校验。
3. 查缓存 (:54-65)：读锁 `lookup(key, &input)`；命中则：重放 mutable 调用 (:56-59) → (lookup 内) `age` 归零 → 返回 `entry.output.clone()`。函数体零执行。

4. **未命中：挂约束** (:67-71) : `input.attach(storage, constraint)` —— **此后所有 tracked 调用同时** `sink.emit(call, ret)` 記入 Constraint。
5. **执行** : `func(input)` **产生** `output` (**此阶段不持锁**)。
6. **写回** (:74-87) : **写锁** `insert(key, constraint, output.clone()) : 0k ; AlreadyExists` (**并发同参数抢先插入 静默忽略**) ; `MissingCall` (**函数非** `reorderably deterministic` , **debug 下 panic** `"comemo: memoized function is non-deterministic"` , **release 下仅不缓存**)。
7. **miss 路径无论插入成败都返回真实计算结果**。

并发结构 : `Cache` 内是 `LazyLock<RwLock<CacheData>>` (`src/memoize.rs:114`) ; `lookup` 走读锁、`insert` 走写锁。递归 `memoize` (`calc` 例子里 `evaluate` 调 `evaluate`) 之所以安全, 是因为 `execute` 阶段不持锁, 写锁仅在 `insert` 的短暂窗口内获取。

CallTree : 四张表 + NodeId 编码

`src/tree.rs:15-26` :

```
CallTree<C, T> {
    inner: Slab<InnerNode<C>>,           // 内部节点: 存调用
    leaves: Slab<LeafNode<T>>,           // 叶子: 直接存输出值
    start: FxHashMap<u128, NodeId>,      // key 哈希 → 起始节点
    edges: FxHashMap<(InnerId, u128), NodeId>, // (父节点, 该调用返回哈希) → 子节点
}
```

- `InnerNode { call, children, parent }` (:29-36) : `children` 归零即删除; `parent` 供自底向上剪枝。
- `NodeId(isize)` 编码 (:234-254) : 非负值 = `inner slab 下标`; 负值 = `-(i) - 1` (即 `~i`) , 解出 `leaf 下标`。两个 Slab 共用一个 `id` 空间而零开销。(为什么不是 `-i` : 否则 `leaf 0` 和 `inner 0` 撞号。)
- 语义 : 一条从 `start[key]` 到叶子的路径 = “key 相同、且依次做过调用 $c_1 \dots c_n$ 、返回哈希为 $r_1 \dots r_n$ ” 的一次历史执行; `edges[(id_i, r_i)]` 是“在第 i 个调用处, 若 `oracle` 给出的返回哈希是 r_i , 就往哪走”。**同一 key 下, 多个调用序列共享公共前缀, 返回哈希不同则分叉。**

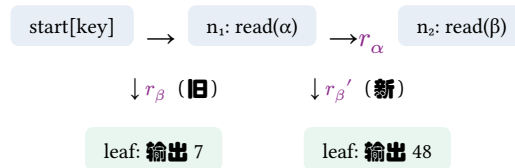


图 3 CallTree 示意: `beta.calc` 修改前后两条执行路径共享前缀、按返回哈希分叉

get : 用 oracle 重放校验

`src/tree.rs:57-72` :

```
cursor = start.get(key)?                // 无起始节点即 miss
loop {
    Leaf(id) => return &leaves[id].value; // 走到叶子 = 命中
    Inner(id) => ret = oracle(&inner[id].call); // 对当前 tracked 实例重放该调用
    cursor = edges.get(&(id, ret))?; // 返回哈希决定走哪条边; 缺边即 miss
}
```

`oracle` 就是 `memoize.rs` 传入的 `|c| input.call(c)` (`src/memoize.rs:156-158`) —— **在当前 tracked 输入上重放历史调用。完整值从未被比较, 只有 128 位返回哈希参与决策。**

insert : 前缀对齐 + 开新路径

`src/tree.rs:80-137`。状态机 : `cursor` (当前树节点) 与 `predecessor: Option<(InnerId, u128)>` (一旦进入“建新路径”模式, 记录上一条待连的边)。

- `predecessor.is_none() && cursor 存在` (:90-111) : 还在沿已有路径走。若当前节点是叶子 → `AlreadyExists` ; 否则 `sequence.extract(call)` 取本次执行对同一调用的返回哈希:
 - `extract` 返回 `None` → `InsertError::MissingCall` (:99-101) : 树路径上的第 $N+1$ 个调用, 本次执行没做过——指向 `memoized` 函数的非确定性 (违反 `reorderably deterministic` , 变体文档 :271-275) ;
 - 有边 → 继续沿路径走; 无边 → 转入建新模式。
- 否则 (:113-127) : `sequence.next()` 取下一个新调用, 插 `InnerNode` , `link` 挂边 (:139-154 : 起始节点写 `start` , 否则写 `edges` 并 `children += 1`) 。
- 收尾 (:130-137) : 整个序列都是已有路径的前缀 → `AlreadyExists` (并发重复插入) ; 否则插 `LeafNode` 并 `link` 。

retain : 自叶向根剪枝

src/tree.rs:158-190 :

- **第一阶段** leaves.retain (:163-178) : 对每个叶子问谓词；删叶子时沿 parent 链向上：父节点 children > 1 则 children -= 1 并停止；否则递归删父、继续向上。
- **第二阶段** (:180-190) : 用 exists 闭包清掉 edges / start 中指向已删节点的悬挂条目。
- **不变量**由 #[cfg(test)] assert_consistency (:193-209) 在测试里逐步校验。

Cache / CacheEntry 与 age 逐出

- CacheEntry { output, mutable: Vec<C>, age: AtomicUsize } (src/memoize.rs:137-144) : age 用 AtomicUsize 是因为命中路径只持读锁却要把 age 归零 (:159)。
- CacheData::evict (:149-154) : 每条 entry age += 1 , 保留 age <= max_age 的。age = 距上次命中经历了多少次逐出, 命中即归零。max_age = 0 等价于清空。
- EVICTORS 全局注册表 (src/memoize.rs:13-14) : 宏为每个 Cache 生成初始化代码时调 register_evictor (:109-111) 挂入该缓存的 evict 函数指针。用户调 comemo::evict(max_age) (:101-106) 遍历所有注册项逐个逐出, 并联动 accelerate::evict()。Typst 每排版一遍文档就调一次 evict(30) , 实现“30 轮没用的结果被淘汰”。

calc 例子完整走一遍

以 evaluate("eval alpha.calc", files.track()) 为例 (examples/calc.rs) :

1. **首次 (miss)** : key = hash("eval alpha.calc") ; start 无此 key → miss → attach Constraint → 执行 : files.read("alpha.calc") emit (read α, h_α) , 递归 evaluate 读取 beta → emit (read β, h_β) → insert 建树 : start[key] → n₁(read α) → h_α → n₂(read β) → h_β → leaf(7) 。
2. **第二次 (未改文件, hit)** : key 相同 → oracle 重放 read(α) 得 h_α → 边存在 → 重放 read(β) 得 h_β → 叶子 → 命中返回 7 , read 函数体没执行。
3. **改了 gamma.calc 后 (仍 hit)** : alpha 的执行链没读过 gamma , 重放哈希照旧 → 命中 (examples/calc.rs:26-29)。
无关文件的改变不会引起重算。
4. **改了 beta.calc 后 (miss)** : oracle 重放 read(β) 得新哈希 h_β' ≠ h_β → edges 查不到 → miss → 重新执行, 新序列以 (read α, h_α) 与前缀对齐、从 read β 处开新路径插入 (examples/calc.rs:31-35)。

缓存失效的粒度是“实际读到的 tracked 数据”, 不是“整个输入”——这正是 constrained memoization 的本质。

本章源码精读路线

建议顺序: memoize.rs → constraint.rs → tree.rs (先主流程建立问题感, 再看记录器, 最后啃数据结构)。

1. src/memoize.rs:17-93 : 参数为何从外部传入 ; 命中路径三件事的顺序 ; AlreadyExists 静默吞掉、MissingCall debug 下 panic。
2. src/memoize.rs:96-111 : 全局逐出如何发现每一个 Cache ; accelerate::evict() 联动。
3. src/memoize.rs:137-180 : age 为何原子 ; oracle 闭包与 tree.rs get 的衔接。
4. src/constraint.rs:67-77 : 双通道分流的一行判断。
5. src/constraint.rs:96-139 : insert / next / extract 分别服务谁 ; debug-only 纯度检查。
6. src/tree.rs:15-26 + :234-254 : 四张表与 NodeId 负值编码。
7. src/tree.rs:57-72 (get) : ? 运算符天然把“无起始节点/缺边”映射为 miss。
8. src/tree.rs:80-137 (insert) : 最难一段。cursor / predecessor 双状态 ; 两个 AlreadyExists 触发点 (:93、:130) ; MissingCall 唯一触发点 (:99)。配合 InsertError 文档 (:264-275) 读。
9. src/tree.rs:158-190 (retain) : children > 1 停止条件 ; inner 为什么不能直接 Slab::retain。
10. src/tree.rs:278-418 (单元测试) : test_call_tree 手工构造共享前缀 + 分叉并断言四张表的精确大小, 是理解结构的最好样例 ; test_ops 维护影子模型做全量比对, 是 quickcheck 范式。

易错点与设计权衡

- MissingCall 的行为分裂 (src/memoize.rs:80-85) : debug panic / release 静默不缓存但结果正确。它几乎总是用户函数在不同轮次对相同 tracked 状态做了不同调用序列, 不是 comemo 自身的 bug。
- 两道纯度防线的分工 : CallSequence::insert 的 panic 抓“同一轮内同一调用返回值不同” ; MissingCall 抓“跨轮调用序列结构不同”。前者靠去重顺带检测, 后者要靠树的形状才能发现。
- 同 key 多路径并存不是缺陷 : 同一 key 下可同时缓存“读过 beta v1”和“读过 beta v2”的执行, 两条路径都可能在后续命中。

- **为什么校验用 128 位返回哈希而非完整值：**(a) tracked 返回值可能很大或不可克隆；(b) oracle 重放只需返回 `u128`，加速器还能让重放近乎 $O(1)$ ；(c) 128 位冲突概率可忽略。代价：理论上哈希碰撞会导致错误命中，comemo 接受这个概率。[推断：权衡理由基于代码与文档语义推断，源码未明文写出。]
- **可变调用不参与校验、只参与重放：**初学者易以为 mutable 调用也会让缓存失效——不会。它不进树、不进 key。若可变方法实际影响了后续只读调用的结果，约束系统无法察觉——这是用户契约。
- **写锁粒度：**insert 持整个 `CacheData` 的写锁，同一函数的并发 miss 串行化；lookup 走读锁可并行。读多写少的负载下这个取舍是对的。

四、过程宏层 (comemo-macros)

定位：零样板体验的来源——编译器把普通函数/impl 块改写为接入运行时的代码

comemo-macros 是独立的 proc-macro crate (`macros/Cargo.toml:14-15` 声明 `proc-macro = true` , 依赖只有 `proc-macro2 / quote / syn` 三件套)。对外只导出两个属性宏：`#[memoize]` (入口 `macros/src/lib.rs:133-139`) 与 `#[track]` (入口 `macros/src/lib.rs:211-216`)。缓存查找、参数打包、调用追踪、约束生成全部在编译期由宏生成；宏本身不含任何运行时逻辑，生成的代码只是对运行时机制 (`internal::memoize`、`Cache`、`Call`、`Surfaces`、`Sink`) 的调用拼装。

`#[memoize]` 的展开流程

`expand` (`macros/src/memoize.rs:6-16`)：要求被标注项是 `syn::Item::Fn`，然后两步：`prepare` (解析+校验) → `process` (代码生成)。

`prepare` (:44-63)：属性参数解析成 `Meta { enabled: Option<syn::Expr> }` (`enabled` 是自定义关键字，:172-174)；逐参数调 `prepare_arg`；无返回值的函数输出规范化为 `()`。

`prepare_arg` (:66-97)：宏层并不区分 `hashed / Tracked / TrackedMut` 三类参数——区分完全推迟到运行时由 `Input` trait 按类型分派。宏只做语法级校验：

- `&mut self` 接收者禁止 (:69-71)；
- 参数模式必须是简单标识符：拒绝解构 (:76-85)；
- `&mut T` 类型的普通参数禁止 (:87-92)。

`process` (:100-170)：生成四组 token 后整体替换函数体：

1. 每个参数一条编译期断言 `assert_hashable_or_trackable(&arg)` (:102-110)，用 `quote_spanned!` 把错误定位到原函数；
2. 参数值元组：`self` 被替换为 `hash(&self)` (:113-119) ——方法场景下 `self` 一律走哈希；
3. 参数类型元组：`self` 的类型槽位记为 `()` (:121-125)；
4. 内层闭包：原函数体原封不动变成闭包体 (:134-137)；
5. 外层函数剥掉参数上的 `mut`，`enabled` 缺省补 `true`，用 `parse_quote!` 换入新函数体 (:149-167)。

生成的代码骨架 (与 `macros/src/memoize.rs:149-167` 的 `parse_quote!` 逐行对应)：

```
fn evaluate(script: &str, files: Tracked<Files>) -> i32 {
    // 每个被 memoize 的函数独占一份函数内 static 缓存
    static __CACHE: ::comemo::internal::Cache<
        <::comemo::internal::Multi<(&str, Tracked<Files>)>
        as ::comemo::internal::Input>::Call,
        i32,
    > = ::comemo::internal::Cache::new(|| {
        // 初始化时把本缓存的驱逐函数挂进全局注册表
        ::comemo::internal::register_evictor(|max_age| __CACHE.evict(max_age));
        ::core::default::Default::default()
    });

    ::comemo::internal::assert_hashable_or_trackable(&script);
    ::comemo::internal::assert_hashable_or_trackable(&files);

    ::comemo::internal::memoize(
        &__CACHE,
        ::comemo::internal::Multi((script, files)),
        &mut ::core::default::Default::default(), // 栈上 (storage, constraint)
        true, // enabled 表达式, 缺省 true (每次调用求值)
        |::comemo::internal::Multi((script, files))| -> i32 { /* 原函数体 */ },
    )
}
```

要点：__CACHE 是函数内 static；Cache 的类型参数用 `<Multi<...> as Input>::Call` ——不同参数类型组合对应不同 Call 类型，由运行时 `Input` trait 计算；`enabled` 表达式在每次调用时求值，为 `false` 时运行时跳过哈希与缓存（动机见文档 `macros/src/lib.rs:108-131`：廉价小函数绕开缓存开销）。

`#[track]` 的展开流程

入口 `macros/src/lib.rs:211-216` → `track::expand` (`macros/src/track.rs:4-63`)。支持两种目标：

- `inherent impl` 块 (:9-28)：拒绝类型泛型与 `const` 泛型（生命周期允许），逐成员 `prepare_impl_method`；

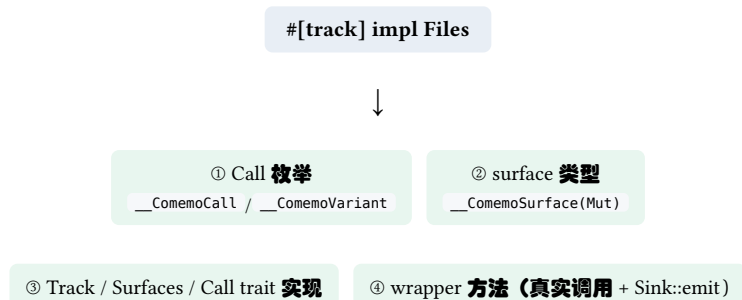
- **trait 定义** (:29-41) : trait 不能有任何泛型参数（连生命周期也不行），类型被改写为 `dyn TraitName + '.__comemo_dynamic` ——trait 场景下追踪对象是 trait object；
- **其他项一律报错** (:42)。

校验链：prepare_method (:100-192) 逐条规则：

规则	行号
不能 <code>unsafe</code> / <code>async</code> / <code>const</code>	:101-111
方法不能有类型/const 泛型（生命周期可以）	:113-120
必须有 self 接收者，且必须按引用（&self / &mut self）	:122-129
参数必须是简单标识符，禁解构、禁 mut 绑定、禁 impl Trait	:141-155
参数不能是 &mut T；&T 记为 Kind::Reference	:156-164
返回值不能是 &mut 引用	:171-176
可变方法（&mut self）不能有返回值	:178-182

最后在 `expand` 末尾做全块级校验：**同一个 impl/trait 不能混合可变与不可变方法** (:45-50)，**生成物包进匿名常量** `const _: () = { ... }` **避免污染命名空间** (:56-62)。

生成物的四个部分：



代码 1 #[track] 宏的四个生成物（create_variants / create / create_call(_mut) / create_wrapper）

1. **Call 枚举**：create_variant (:333-337) **为每个方法生成元组变体** 方法名(<参数类型 as ToOwned>::Owned, ...) ——参数一律存 owned 形式。外包 `pub struct __ComemoCall(__ComemoVariant)` (:214-215)，并实现 `Call::is_mutable()`。两者 derive `Clone, PartialEq, Hash` ——这就是调用记录可哈希、可比较的来源。
2. **surface 类型**：#[repr(transparent)] `pub struct __ComemoSurface<'t, ...>(Tracked<'t, T>)` 与 `__ComemoSurfaceMut` (:312-328)。它们是 `Tracked` 解引用后的目标类型，上面挂着 wrapper 方法。
3. **Track / Surfaces / Call 三个 trait 实现** (:267-310)：Track::call 逐变体 match (create_call ,:340-357)：**不可变方法真实调用并把返回值哈希成 u128，可变方法直接返回 0**；Track::call_mut (create_call_mut ,:360-376)：**可变方法真实执行，不可变方法什么都不做。这两个方法就是运行时“重放”的落点**。Surfaces 实现用 `repr(transparent)` 保证布局一致后直接指针强转 (unsafe ,:291-308)。
4. **wrapper 方法**：create_wrapper (:379-409) 在 surface 类型上生成与原方法同签名的方法，按三种情形选择折包函数 (to_parts_ref / to_parts_mut_ref / to_parts_mut_mut)：

```

let (value, sink) = ::comemo::internal::to_parts_ref(self.0);
if let Some(sink) = sink { // 处于被追踪的 memoized 调用中
    let variant = __ComemoVariant::read(path.to_owned());
    let output = value.read(path); // 先真实调用
    ::comemo::internal::Sink::emit(sink, __ComemoCall(variant),
        ::comemo::internal::hash(&output));
    output
} else { // 未被追踪：直接调用，零开销
    value.read(path)
}

```

utils.rs 的辅助

`parse_key_value<K, V>` (macros/src/utils.rs:9-21)：用 `peek` 试探关键字，存在则解析 `key = value` ——支撑 `#[memoize(enabled = ...)]` 的语法。eat_comma (:24-28)：吃掉可选逗号。

手写等价展开示例（示意）

以 `examples/calcul.rs` 的 `#[track] impl Files { fn read(&self, path: &str) -> String }` 为蓝本（示意，省略泛型与 `where` 子句细节）：

```
const _: () = {
    // ① Call 枚举 (create_variant, track.rs:333-337)
    #[derive(Clone, PartialEq, Hash)]
    enum __ComemoVariant { read(<&str as ToOwned>::Owned /* = String */) }
    #[derive(Clone, PartialEq, Hash)]
    pub struct __ComemoCall(__ComemoVariant);
    impl ::comemo::internal::Call for __ComemoCall {
        fn is_mutable(&self) -> bool { match &self.0 { __ComemoVariant::read(..) => false } }
    }

    // ② Track 实现 (create_call / create_call_mut, track.rs:340-376)
    impl ::comemo::Track for Files {
        type Call = __ComemoCall;
        fn call(&self, call: &Self::Call) -> u128 {
            match call.0 { __ComemoVariant::read(ref path) =>
                ::comemo::internal::hash(&self.read(path)) } // 真实调用 + 哈希返回值
        }
        fn call_mut(&mut self, call: &Self::Call) {
            match call.0 { __ComemoVariant::read(..) => {} } // 不可变方法：空操作
        }
    }

    // ③ surface 类型 + ④ Surfaces 实现 (track.rs:283-328, repr(transparent) 指针强转)
    #[repr(transparent)]
    pub struct __ComemoSurface<'t>(<'t, ::comemo::Tracked<'t, Files>);
    // ⑤ wrapper 方法 (create_wrapper, track.rs:379-409): 真实调用 + Sink::emit
};
```

`#[memoize]` 侧的展开见上文“生成的代码骨架”。

本章源码精读路线

1. `macros/src/lib.rs:23-131` (memoize 宏文档)：先读。三类参数与四条限制是理解运行时的前提。
2. `macros/src/memoize.rs:6-16`：入口与分派；memoize 也可用于方法（impl 块里的方法经 `syn::Item` 解析后同样是 Fn）。
3. `macros/src/memoize.rs:44-97`：读校验逻辑，对照文档限制找对应 `bail!`。
4. `macros/src/memoize.rs:100-170` (process)：本章核心。重点：self 为什么被换成 `hash(&self)`；static `__CACHE` 的类型参数为什么用 `<Multi<...> as Input>::Call`；`register_evictor` 把每个函数的缓存挂进全局驱逐体系。
5. `macros/src/track.rs:4-63` (expand)：两种目标分派与“禁混合可变性”总开关：`const _: () = {}` 防名字污染。
6. `macros/src/track.rs:100-192` (prepare_method)：对照文档逐条看校验；`Kind::Reference` 判定决定后面 `to_owned()` 加不加。
7. `macros/src/track.rs:195-229` 与 `:232-330`：生成物全景；`repr(transparent)` + 指针强转是 surface 零成本的原因。
8. `macros/src/track.rs:333-409` (四个 `create_*`)：与“四部分”一一对应；`create_wrapper` 中 `sink` 为 `None` 的快路径说明未追踪时开销为零。
9. `macros/src/utils.rs:9-28`：一分钟扫完。

易错点与设计权衡

- 为什么 `tracked` 方法参数要 `ToOwned`：调用记录要活下来。每次 `tracked` 调用被存入 `__ComemoVariant` 枚举，进而进入缓存的约束结构，生命周期远长于方法调用本身——引用参数必须转成 `owned` 才能存。`Kind::Reference` 分支在真实调用时把 `owned` 值再借回去，避免无谓克隆。
- 为什么可变 `tracked` 方法无返回值：重放语义决定。命中时不执行函数体，只通过 `Track::call_mut` 重放历史变更；若可变方法有返回值，调用方拿到的将是上次缓存的值，语义无法保证。
- 为什么不能混合可变与不可变方法：`Surfaces` 实现只有一份。如果同一类型既能不可变追踪又能可变追踪，不可变追踪路径会漏记对状态的依赖，缓存命中时状态却已变化——作者干脆一刀切：要么全不可变（用 `Tracked`），要么全可变（用 `TrackedMut`）。
- `memoize` 宏层不区分三种参数是刻意的：参数是 `hashed` 还是 `tracked` 完全由类型在运行时经 `Input` trait 分派，宏只做语法校验。这让同一套宏代码覆盖三类参数。
- `&mut` 参数被禁不是能力问题而是纯度保证：普通 `&mut T` 的变更无法被追踪/重放，宏在三处拦截（`macros/src/memoize.rs:69-71`、`:87-92`、`macros/src/track.rs:157-158`）。
- `enabled` 表达式放在运行时求值：牺牲一次布尔求值换来“同一签名、可选缓存”，对 `Typst` 中大量廉价小函数是关键优化。

五、示例、测试体系与源码阅读指南

定位：示例与测试是 comemo 的“活文档”——先读示例建立直觉，再用测试反查精确语义

examples/basic.rs (37 行) 展示无跟踪的纯 memoization 最小用法；examples/calc.rs (67 行) 用玩具脚本语言演示增量计算的核心价值；tests/tests.rs (733 行) 把每个公开机制固化为可执行的行为契约，其中 test! 宏 (tests/tests.rs:12-21) 直接断言“这次调用是 Hit 还是 Miss”。支撑这一断言能力的是 testing feature。

examples/basic.rs 逐行讲解（最小上手例子）

只用到 #[memoize]，不涉及 Tracked 输入，证明 comemo 退化为普通 memoizer 也能用。

- empty(); // [Miss] (examples/basic.rs:7)：缓存为空，首次必 miss；第 8、9 行 [Hit]：无参数，键恒定，此后永远命中。
- double(2) miss → double(4) miss (参数不同 → 键不同) → double(2) hit (:11-13)。
- sum(2, 3) 与 sum(4, 2) 是不同的键 (:15-18) ——memoize 不做参数交换归一化。
- 要点：每个被标注的函数拥有独立的静态缓存，无需任何结构改造。

examples/calc.rs 的 Hit/Miss 逐条分析

calc 语言支持 + 加法和 eval <path> 引用其他文件。evaluate 被 #[memoize] 标注 (examples/calc.rs:40-41)，递归调用自身求值子脚本；Files 是被跟踪的输入，read 经 #[track] (:56-59) 成为受约束的访问点，write 是普通可变方法 (:65-68)。

1. 第 1 次 (:18-19) [Miss] → 7：缓存为空，顶层必 miss。执行中 alpha 通过 files.read 读 beta (约束：beta 的内容哈希)，递归求值得 7。
2. 第 2 次 (:21-23) [Miss] → 5：顶层键 "eval beta.calc" 从未出现过 → 顶层 miss。但递归调用 evaluate("2 + 3", files) 的键与约束和第 1 次完全一致，内层命中 beta 的求值没有重跑——顶层 miss ≠ 全部重算。
3. 第 3 次 (:28-29) [Hit] → 7：gamma.calc 被改写，但 alpha 的执行只约束了 beta 的内容，gamma 不在约束集合里 → 顶层直接命中。无关文件的改变不会引起重算。
4. 第 4 次 (:34-35) [Miss] → 48：beta 被改写为 "4 + eval gamma.calc"，约束包含 beta 的旧内容哈希，重验失败 → miss 并重新执行，得 2 + (4 + 42) = 48。

顶层 Hit/Miss 由“参数键 + 上次执行留下的约束集合是否仍满足”决定；子表达式的复用来自递归调用各自独立的缓存条目。

src/testing.rs 与 testing feature 的联动

- src/testing.rs (21 行) 是线程局部布尔标志：LAST_WAS_HIT: Cell<bool> (:3-5)，公开读取器 last_was_hit() (:9-11)，内部写入器 register_hit() / register_miss() (:14-21)。
- 写入点在 src/memoize.rs：enabled 旁路 (:41-42) 与重算出口 (:88-89) register_miss()；命中路径 (:61-62) register_hit()。三处都用 #[cfg(feature = "testing")] 门控，生产构建零开销。
- testing = [] 是空 feature (Cargo.toml:46)；[[test]] required-features = ["macros", "testing"] (Cargo.toml:61-64) 意味着裸 cargo test 会静默跳过整个集成测试套件——必须 cargo test --all-features。
- 测试侧的 test! 宏 (tests/tests.rs:12-21) 先断言结果值，再断言 last_was_hit()，把命中/未命中变成一等测试断言。thread_local + #[serial] 避免并行测试串扰（缓存是函数级静态全局状态）。

tests/tests.rs 测试版图

共 13 个 #[test] + 1 个 #[quickcheck]，全部 #[serial]：

类别	测试函数	覆盖的行为契约
基础 memoize	test_basic :26	空参/多参/递归 fib 的 Hit-Miss 序列；大计算只算一次
跟踪式增量	test_calc :89	calc 全场景，比 example 多 46/48/86 三步演进 (:96-104)

缓存逐出	test_evict :123	evict(2) 需多次调用才逐出(分代), evict(0) 立即全清
跟踪 trait 对象	test_tracked_trait :149	Tracked<dyn Loader> \n#[track] 用于 trait
memoize 方法	test_memoized_methods :181	&self 方法与按值 self 方法
参数形态	test_kinds :207	返回值借用 self/参数; tracked 返回值带生命周期; 空 impl 合法
生命周期与链式	test_lifetime :284 / test_chain :312 / test_variance :339	带生命周期的被跟踪类型; 链式结构前缀共享时 Hit ; Tracked<T> 对 T 协变
可变跟踪重放	test_purely_mutable :377 / test_mutable_nested :414	TrackedMut 的 emit 在命中时被重放; 嵌套 memoize 间传递 TrackedMut 一起重放
同键多约束	test_many_with_same_key :442	同一键下多条不同约束条目, 重验能挑出仍成立的那条
确定性检查	test_non_deterministic :486 / test_deterministic_out_of_order :516	约束依赖路径的非确定函数 debug 下 panic ;乱序但总体确定的函数允许
纯度检查	test_impure_tracked_method :569	tracked 方法内部有副作用时 debug 构建 panic
条件禁用	test_with_disabled :593	#[memoize(enabled = ...)] ; 小输入不缓存, 大输入正常
性质测试	test_memoize_quickcheck :607	随机输入下 memoize 版与朴素直算版结果与副作用完全一致

提示: quickcheck 验证的不变量不是性能, 而是**语义等价性**——任意随机输入下, memoize 版(含 Tracked 读取与 TrackedMut 变更重放)与朴素直算版产生相同返回值和相同副作用。这是整个 crate 最重要的性质: 缓存不可观察。

注意: 没有真正的多线程并发测试; 最接近的是 test_deterministic_out_of_order , 其注释明确“内部使用多线程的确定性函数”是被支持的场景 (tests/tests.rs:511-513)。

源码阅读指南: 三遍阅读法 (本指南的压轴)

第一遍: 文档与示例——建立“它做什么”的直觉

路径: README.md → examples/basic.rs → examples/calc.rs。

- **目标:** 理解 #[memoize] / #[track] 的表面语法, 以及 Hit/Miss 的判定直觉 (参数键 + 约束重验)。
- **预期收获:** 能解释 calc.rs 四个 assert 各自的 Hit/Miss 原因。
- **动手验证:** cargo run --example calc ; 给 calc.rs 加一个自己的 assert (比如改 gamma 后再 eval alpha), 用直觉预测 Hit/Miss 再跑一遍对照。

第二遍: 运行时核心——理解“它怎么做”

路径: src/lib.rs (模块导览与公开 API, :99-103 看 feature 门控与 re-export) → src/track.rs → src/input.rs → src/constraint.rs → src/tree.rs → src/memoize.rs (:17-93 是核心路径) → src/accelerate.rs → src/hash.rs。

- **目标:** 能说清一次 memoized 调用的完整生命周期: 查键 → 约束重验 → 命中返回 / 重算并记录新约束。
- **卡住时的调试建议:**
 - 开一个最小复现(复制 basic.rs 结构), 开 testing feature 后在每次调用后打印 comemo::testing::last_was_hit() , 观察预期与实际的分歧点;

- 注意大量正确性检查（非确定性、不纯跟踪方法）只在 `debug_assertions` 下启用——`release` 构建行为不同，调试时别用 `--release` ；
- 读 `memoize.rs` 时对照 `tests.rs` 的 `test!` 序列，每条 Hit/Miss 都能在 `memoize.rs` 里找到对应分支。

第三遍：宏与测试——理解“糖怎么化”与“契约是什么”

路径：`macros/src/lib.rs` → `macros/src/memoize.rs` → `macros/src/track.rs` → `macros/src/utils.rs` → `tests/tests.rs`（按测试版图挑感兴趣的读）。

- 目标：把第二遍看到的运行时 API 与宏生成的代码对上号；知道每个公开行为有哪个测试兜底。
- 反查契约的方法：想确认某机制的语义时，先 `grep` 测试函数名（如 `grep -n "fn test_evict" tests/tests.rs`），读它的 `test!` 序列——那是该机制最精确、可执行的规格说明；再回运行时源码找实现。遇到边缘行为，`should_panic` 消息（`tests/tests.rs:485`）往往比文档更权威。

易错点

- 裸 `cargo test` 会“假绿”：必须 `cargo test --all-features`（CI 也是这么跑的）。
- `last_was_hit` 是线程局部 + 串行假设：自己写测试时别忘了 `#[serial]`。
- 示例注释的“Miss”有歧义风险：`calc.rs` 第 2 个 `[Miss]` 指顶层 miss，但内层 `beta` 求值是命中的。读测试版 `test_calc` 更精确。
- `debug` 与 `release` 的行为差异：非确定性检查与不纯方法检查只在 `debug_assertions` 下存在。这是刻意的性能权衡：正确性护栏是开发期工具，运行时零开销；代价是 `release` 下非确定性函数会静默产生无效缓存。
- 逐出不是即时的：`evict(2)` 要连续调用多次才生效——逐出按“代”工作；`evict(0)` 才是立即全清。读 `evict` 实现前先看 `test_evict`，否则容易误判为 bug。

附录 A：关键类型与函数速查表

A.1 公开 API（用户侧）

名称	位置	作用
<code>#[memoize]</code>	<code>macros/src/lib.rs:133</code>	属性宏：把函数包装为受约束记忆化函数；支持 <code>enabled = <expr></code>
<code>#[track]</code>	<code>macros/src/lib.rs:211</code>	属性宏：为 <code>impl</code> 块/trait 实现 <code>Track</code> ，使方法调用可被跟踪
<code>Tracked<'a, T></code>	<code>src/track.rs:162</code>	不可变跟踪容器， <code>Deref</code> 到宏生成的 <code>surface</code> 类型
<code>TrackedMut<'a, T></code>	<code>src/track.rs:223</code>	可变跟踪容器，命中时变更被重放
<code>Track trait</code>	<code>src/track.rs:12</code>	<code>track()</code> / <code>track_mut()</code> / <code>track_with()</code> / <code>track_mut_with()</code>
<code>Constraint<C></code>	<code>src/constraint.rs:16</code>	约束：记录一次执行的调用序列， <code>validate</code> 验证是否仍成立
<code>evict(max_age)</code>	<code>src/memoize.rs:101</code>	全局缓存驱逐：清除 <code>age</code> 超过 <code>max_age</code> 的条目； <code>evict(0)</code> 全清
<code>testing::last_was_hit()</code>	<code>src/testing.rs:9</code>	测试断言入口（需 <code>testing</code> feature）

A.2 运行时核心（internal）

名称	位置	作用
<code>memoize()</code>	<code>src/memoize.rs:17</code>	主调度：旁路 → key 哈希 → lookup/重放 → attach/执行 → insert
<code>Cache<C, Out></code>	<code>src/memoize.rs:114</code>	单函数缓存： <code>LazyLock<RwLock<CacheData>></code> ， <code>const fn new</code>
<code>CacheEntry</code>	<code>src/memoize.rs:137</code>	output + 待重放 mutable 调用 + age (AtomicUsize)
<code>register_evictor</code>	<code>src/memoize.rs:109</code>	宏生成代码把每个 <code>Cache</code> 的 <code>evictor</code> 注册进全局表
<code>CallTree<C, T></code>	<code>src/tree.rs:15</code>	四表结构：inner/leaves Slab + start/edges FxHashMap
<code>CallTree::get / insert / retain</code>	<code>src/tree.rs:57 / :80 / :158</code>	oracle 重放查询 / 前缀对齐插入 / 自叶向根剪枝
<code>NodeId(isize)</code>	<code>src/tree.rs:234</code>	非负=inner 下标，负= ~i =leaf 下标
<code>InsertError</code>	<code>src/tree.rs:264</code>	<code>AlreadyExists</code> / <code>MissingCall</code> （非确定性信号）
<code>CallSequence<C></code>	<code>src/constraint.rs:82</code>	去重调用序列： <code>vec</code> + <code>map</code> + <code>cursor</code> ； <code>insert</code> / <code>next</code> / <code>extract</code>
<code>Input<'a> trait</code>	<code>src/input.rs:13</code>	输入抽象： <code>Call/Storage/key/call/call_mut/attach</code> 五件套
<code>MergedSink</code>	<code>src/input.rs:156</code>	串联嵌套调用的 <code>sink</code> 链；emit 短路传播
<code>Multi<T> / multi!</code>	<code>src/input.rs:180 / :182</code>	0-12 元组输入的 <code>Input</code> 实现
<code>Sink / Call / Surfaces</code>	<code>src/track.rs:59 / :79 / :92</code>	事件汇 / 调用抽象 / <code>surface</code> 类型族
<code>accelerate::{id, get, evict}</code>	<code>src/accelerate.rs:18 / :36 / :24</code>	约束验证加速器：单调 ID + epoch offset
<code>hash::hash</code>	<code>src/hash.rs:7</code>	<code>SipHasher13</code> 128 位预哈希，全库统一原语

A.3 宏展开关键点

名称	位置	作用
memoize::expand	macros/src/memoize.rs:6	prepare (校验) → process (生成)
process	macros/src/memoize.rs:100	生成 static __CACHE + internal::memoize 调用
track::expand	macros/src/track.rs:4	分派 impl/trait ; 禁混合可变性; const _: () 包裹
prepare_method	macros/src/track.rs:100	全部方法级校验
create_variants	macros/src/track.rs:195	__ComemoCall / __ComemoVariant 与 is_mutable
create	macros/src/track.rs:232	Track/Surfaces 实现与 surface 类型
create_call(_mut)	macros/src/track.rs:340 / :360	重放落点: 真实调用 + 哈希 / 真实执行副作用
create_wrapper	macros/src/track.rs:379	wrapper 方法: 拆包 → 真实调用 → Sink::emit

附录 B：术语表

术语	含义
memoization (记忆化)	缓存函数返回值，同一组参数只执行一次
constrained memoization (受约束记忆化)	缓存条目同时绑定参数哈希与执行期间观察到的访问记录；被观察部分不变即可命中
incremental computation (增量计算)	输入变化后只重算受影响部分的计算范式
access tracking (访问跟踪)	把 tracked 值上的方法调用记录为 (调用, 返回哈希) 事件
constraint (约束)	一次执行留下的调用序列记录，用于下次调用的命中验证
mutation replay (变更重放)	缓存命中时把记录的可变调用重新应用到参数上
call tree (调用树)	以 (调用, 返回哈希) 序列为路径的前缀树，存储缓存条目
oracle (神谕)	约束验证时对当前 tracked 输入重放历史调用的函数
surface 类型	#[track] 宏生成的包装类型，Tracked 的 Deref 目标，拦截所有方法调用
sink (事件汇)	接收 tracked 调用事件的目的地（如 Constraint）
accelerator (加速器)	缓存“调用哈希 → 返回哈希”，避免约束验证时重复执行昂贵方法
reorderably deterministic (可重序确定)	两次执行中，前缀匹配的调用序列必须包含彼此的全部调用（顺序可不同）
age / eviction (代龄/逐出)	条目距上次命中经历的逐出次数；evict(max_age) 清除超龄条目

—— 全文完 ——

愿这份指南帮你啃下 comemo 的每一行代码。

最后修订：2026 年 8 月 1 日